

Why Zero-Centered Hidden Activations Help Gradient Descent

From sigmoid vs. tanh to gradient geometry, zig-zagging, and conditioning

Paul Agron

September 12, 2026

Abstract

Two activation functions can have the same nonlinear shape, the same derivative profile, and the same representational power, yet train differently under gradient descent. A particularly clean example is obtained by comparing

$$f(z) = 2\sigma(z) \in (0, 2), \quad g(z) = 2\sigma(z) - 1 \in (-1, 1),$$

where $\sigma(z) = 1/(1 + e^{-z})$. The two functions differ only by a vertical translation, $g = f - 1$. If the following layer has a bias, that translation can be absorbed exactly into the bias, so the two networks can represent the same functions. Nevertheless, the centered version often gives gradient descent a more favorable parameterization. The central intuition, emphasized in Stanford CS231n, is that strictly positive hidden activations constrain the signs of per-example weight gradients, which can force a zig-zag optimization path. A complementary, more formal view shows that nonzero activation means couple downstream weights to the bias and create off-diagonal terms in the local curvature matrix. This note derives both viewpoints, explains their relationship, and states the limits of the argument. Appendices discuss subdivision-based saturating activations and modern normalization layers.

Contents

1	The question: if the shape is the same, why should centering matter?	3
2	With a following bias, the two activations have the same representational power	4
3	Backpropagation exposes the key sign effect	4
3.1	Anatomy of one downstream neuron	5
3.2	The scalar backpropagated error signal δ	5
3.3	Gradient with respect to the incoming weights	6
3.4	Why the sign of the hidden activation matters	6
3.5	Why δ itself may be positive or negative	7
4	The zig-zag phenomenon	7
5	A second intuition: zero mean separates “baseline” from “variation”	8

6	Advanced / Optional: The Hessian view	9
6.1	A numerical example	10
7	Why random initialization strengthens the case for centering	10
8	Why this is a preference, not a universal law	11
8.1	Saturation can dominate centering	11
8.2	Range does not guarantee empirical centering	11
8.3	Normalization can change the comparison	11
8.4	Output layers have different semantics	12
9	Practical rule of thumb	12
10	Recommended educational sources	12
A	Subdivision-based and spline-based saturating activation functions	13
A.1	From corner cutting to smooth transitions	13
A.2	A quadratic spline activation	13
A.3	Why “without altering outcomes” matters	14
B	Normalization layers and their relation to centering	14
B.1	Batch Normalization	14
B.2	Layer Normalization	15
B.3	RMSNorm: an important qualification to the centering story	15
B.4	What normalization changes in the $(0, 2)$ vs. $(-1, 1)$ comparison	15

1 The question: if the shape is the same, why should centering matter?

Consider two hidden-layer activation functions

$$f(z) = 2\sigma(z), \quad g(z) = 2\sigma(z) - 1, \quad (1)$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

They have the same S-shape and the same derivative:

$$f'(z) = g'(z) = 2\sigma(z)(1 - \sigma(z)). \quad (2)$$

The only difference is the constant vertical shift

$$g(z) = f(z) - 1. \quad (3)$$

Thus f has range $(0, 2)$, whereas g has range $(-1, 1)$.

The centered function is also a scaled-input hyperbolic tangent:

$$g(z) = 2\sigma(z) - 1 = \tanh\left(\frac{z}{2}\right). \quad (4)$$

This identity is useful because it isolates *centering* from other differences. Comparing the usual $\sigma(z)$ to the usual $\tanh(z)$ changes both vertical centering and horizontal scale. By contrast, f and g in eq. (1) have identical slope profiles and differ only by a constant.

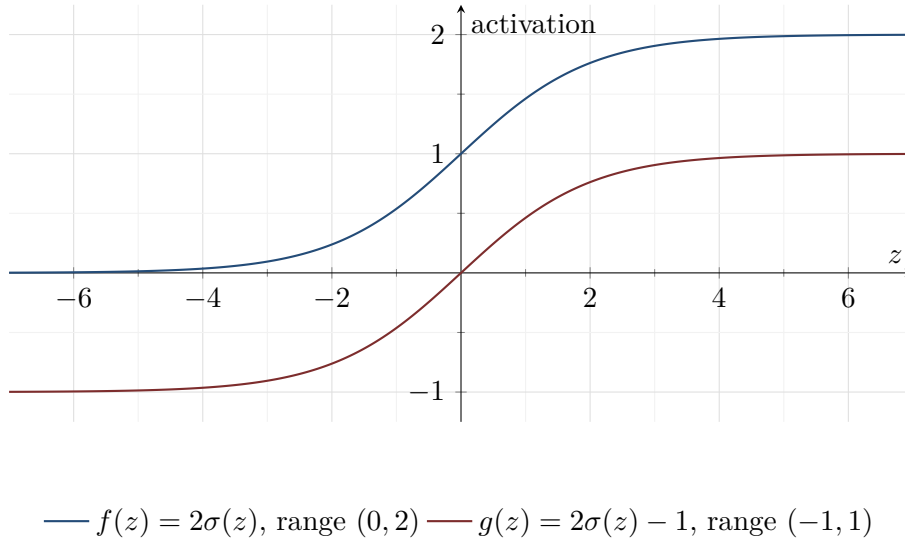


Figure 1: Two activations with identical nonlinear shape and identical derivative, differing only by a vertical translation.

The central question is therefore not one of expressive power: *why can a mere coordinate shift make ordinary gradient descent easier?*

2 With a following bias, the two activations have the same representational power

Take one hidden activation h feeding a downstream affine unit

$$a = vh + b. \quad (5)$$

If the hidden layer uses f , then

$$a_A = vf(z) + b. \quad (6)$$

If it uses $g = f - 1$, then choose a new downstream bias $\tilde{b} = b + v$. The downstream preactivation becomes

$$a_B = vg(z) + \tilde{b} \quad (7)$$

$$= v(f(z) - 1) + (b + v) \quad (8)$$

$$= vf(z) + b \quad (9)$$

$$= a_A. \quad (10)$$

Thus the two networks compute exactly the same downstream preactivation for every input.

For a vector hidden layer $h \in \mathbb{R}^m$, let

$$\tilde{h} = h - c\mathbf{1}.$$

A following affine layer is

$$a = Wh + b.$$

Then

$$W\tilde{h} + \tilde{b} = W(h - c\mathbf{1}) + \tilde{b}. \quad (11)$$

Choosing

$$\tilde{b} = b + cW\mathbf{1} \quad (12)$$

again gives the identical value $Wh + b$.

Representational conclusion. If a hidden activation is changed only by adding or subtracting a constant, and the following layer has a bias, the constant can be absorbed exactly into that bias. Therefore centering does not, by itself, enlarge the set of functions the network can represent. Its advantage is an *optimization* advantage.

3 Backpropagation exposes the key sign effect

This section introduces the one piece of backpropagation notation needed for the zero-centering argument. The goal is not to give a complete treatment of backpropagation, but to make the sign logic fully explicit.

3.1 Anatomy of one downstream neuron

Consider one neuron receiving the hidden activations

$$h_1, h_2, \dots, h_m$$

from the previous layer. Collect them into the incoming activation vector

$$\mathbf{h} = \begin{pmatrix} h_1 \\ h_2 \\ \vdots \\ h_m \end{pmatrix}. \quad (13)$$

The corresponding incoming weights are

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{pmatrix}. \quad (14)$$

The neuron first forms its preactivation

$$z = \mathbf{w}^T \mathbf{h} + b = \sum_{j=1}^m w_j h_j + b, \quad (15)$$

and then applies its activation function

$$y = \phi(z). \quad (16)$$

Thus h_j is simply the activation arriving along the connection whose weight is w_j .

3.2 The scalar backpropagated error signal δ

For one fixed neuron, one fixed training example, and the current parameter values, define

$$\boxed{\delta \equiv \frac{\partial L}{\partial z}}. \quad (17)$$

Here δ is a *scalar value*: it measures how sensitive the final loss is to a small change in this neuron's preactivation z .

The same scalar δ is used for all of the incoming weights of this particular neuron. It is not, however, a global constant: it generally changes from one training example to another, from one neuron to another, and as the network parameters change during training.

By the chain rule,

$$\delta = \frac{\partial L}{\partial z} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} = \frac{\partial L}{\partial y} \cdot \phi'(z). \quad (18)$$

This makes clear why δ may be either positive or negative: its sign depends on the downstream loss gradient and on the current network state.

3.3 Gradient with respect to the incoming weights

For the scalar weight w_j , the chain rule gives

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial w_j} \quad (19)$$

$$= \delta \cdot h_j, \quad (20)$$

because $\partial z / \partial w_j = h_j$ from eq. (15). Likewise,

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial b} = \delta \cdot 1 = \delta. \quad (21)$$

Interpretation. For one incoming connection,

$$\underbrace{\frac{\partial L}{\partial w_j}}_{\text{weight gradient}} = \underbrace{\delta}_{\text{error signal}} \cdot \underbrace{h_j}_{\text{incoming activation}}.$$

Collecting all incoming-weight derivatives into one vector gives

$$\nabla_{\mathbf{w}} L = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial w_m} \end{pmatrix} = \delta \cdot \mathbf{h}. \quad (22)$$

The notation $\nabla_{\mathbf{w}} L$ means the gradient with respect to *all incoming weights of the current neuron*, not with respect to every weight in the entire neural network.

3.4 Why the sign of the hidden activation matters

Equation (22) is the heart of the Stanford CS231n intuition. If the previous hidden layer uses an activation whose output is always positive, such as a logistic sigmoid, then

$$h_j > 0 \quad \text{for every } j.$$

For one training example, the single scalar δ therefore determines the sign of every incoming-weight gradient:

$$\text{sign}\left(\frac{\partial L}{\partial w_j}\right) = \text{sign}(\delta) \quad \text{for every } j. \quad (23)$$

Consequently, all components of $\nabla_{\mathbf{w}} L$ are positive when $\delta > 0$, or all are negative when $\delta < 0$. In two dimensions, the gradient can therefore point only into an all-positive or all-negative quadrant for that example.

If the hidden activations are centered and may take both signs, the products $\delta \cdot h_j$ need not share a sign. A single example can immediately produce a mixed-sign gradient such as

$$(+, -, +, -, \dots),$$

which is impossible when every h_j is positive.

3.5 Why δ itself may be positive or negative

The fact that a sigmoid output y is positive does *not* imply that $\partial L/\partial y$, and hence δ , must be positive. For squared error,

$$L = \frac{1}{2}(y - t)^2, \quad \frac{\partial L}{\partial y} = y - t.$$

If $y = 0.3$ and $t = 1$, then $\partial L/\partial y = -0.7$. If $y = 0.7$ and $t = 0$, then $\partial L/\partial y = +0.7$. Activations describe the current numerical state; gradients describe which infinitesimal direction increases the loss.

For a hidden neuron, the upstream gradient is itself a weighted combination of downstream error signals, so it can again have either sign. Positivity of the incoming h_j 's does not fix the sign of δ ; rather, it makes *all incoming-weight gradient components inherit the same sign from that one scalar* for a single example.

4 The zig-zag phenomenon

Stanford CS231n makes the sign restriction geometrically intuitive. Consider only two incoming weights, w_1, w_2 , of the current neuron. With positive hidden inputs $h_1, h_2 > 0$, the per-example gradient is

$$\nabla_{\mathbf{w}} L = \delta \cdot \mathbf{h} = \delta \cdot \begin{pmatrix} h_1 \\ h_2 \end{pmatrix}.$$

Hence the gradient must have same-sign components. The gradient-descent update

$$\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L$$

has the opposite signs, but still has same-sign components; it therefore also lies in one of the same two opposite quadrants.

Suppose the optimization would benefit from increasing w_1 while decreasing w_2 . That is a mixed-sign displacement. A single-example update generated from strictly positive inputs cannot point directly in that direction. Across successive examples, the scalar δ may change sign; alternating same-sign steps can then produce a mixed-sign *net* displacement, but the path can zig-zag rather than move directly.

This argument should be interpreted carefully:

- It is sharpest for a *single training example* or stochastic-gradient step.
- With a minibatch, gradients are sums

$$\frac{\partial L}{\partial w_j} = \sum_i \delta_i \cdot h_{ij},$$

and different examples can partially cancel. Stanford explicitly notes that minibatches help.

- The argument is an optimization heuristic, not a proof that every positive activation causes slow training.

Nevertheless, it cleanly explains why translating an otherwise identical activation from $(0, 2)$ to $(-1, 1)$ can change the geometry of SGD updates even though the network's representational power is unchanged.

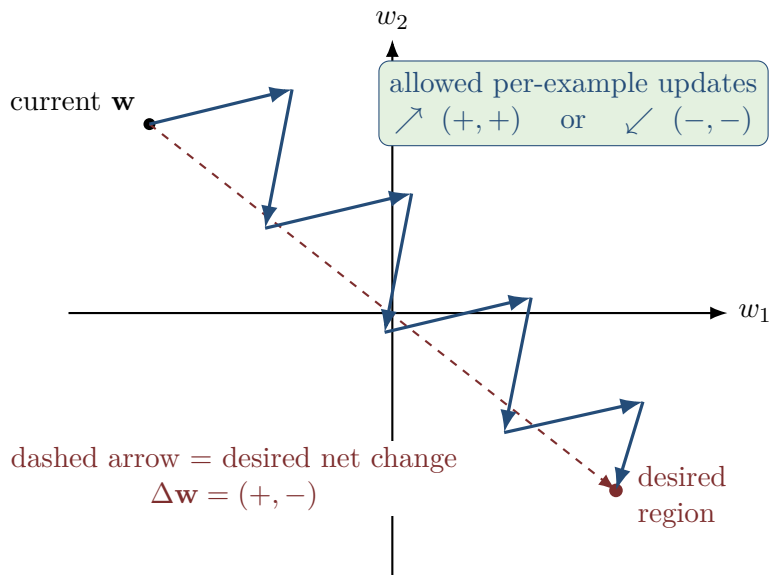


Figure 2: Corrected schematic of the CS231n “zig-zag” intuition. The blue arrows are gradient-descent updates, $\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L$. Every individual blue arrow has same-sign components—it points northeast $(+, +)$ or southwest $(-, -)$ relative to its own tail. Nevertheless, alternating such steps can accumulate to the mixed-sign net displacement indicated by the dashed red arrow. The picture is illustrative, not a literal trajectory for every loss surface.

5 A second intuition: zero mean separates “baseline” from “variation”

The sign argument is local and per-example. A complementary viewpoint appears when one asks how the mean of a hidden feature interacts with the next layer’s bias.

Write a scalar hidden feature as

$$h = \mu + \tilde{h}, \quad \mathbb{E}[\tilde{h}] = 0. \quad (24)$$

A downstream affine model is

$$a = wh + b \quad (25)$$

$$= w(\mu + \tilde{h}) + b \quad (26)$$

$$= w\tilde{h} + (b + w\mu). \quad (27)$$

If $\mu \neq 0$, changing w also changes the effective baseline $b + w\mu$. Thus the weight has two intertwined roles:

1. responding to variation $\tilde{h}$, and
2. shifting the baseline through $w\mu$.

If $\mu = 0$, these roles are naturally separated: w controls sensitivity to the centered feature and b controls the baseline.

This is why the useful property is more precisely

$$\boxed{\mathbb{E}[h] \approx 0,} \quad (28)$$

not merely “the mathematical range includes negative numbers.” A function with range $(-1, 1)$ can still produce activations with mean 0.8 if most preactivations lie on its positive side.

6 Advanced / Optional: The Hessian view

Advanced / optional section. The main zero-centering argument does not require Hessians. This section gives a more formal curvature/conditioning interpretation for readers who want the optimization mathematics. It may be skipped without affecting the later practical conclusions.

The previous intuition can be made exact in a simple model. Consider

$$\hat{y} = wh + b \quad (29)$$

with squared loss

$$L = \frac{1}{2}(\hat{y} - y)^2. \quad (30)$$

For one observation, the second-derivative matrix with respect to (w, b) is

$$\begin{pmatrix} h^2 & h \\ h & 1 \end{pmatrix}.$$

Averaging over the data gives

$$\boxed{H = \begin{pmatrix} \mathbb{E}[h^2] & \mathbb{E}[h] \\ \mathbb{E}[h] & 1 \end{pmatrix}.} \quad (31)$$

The off-diagonal term is exactly $\mathbb{E}[h]$. If the feature is centered,

$$\mathbb{E}[h] = 0, \quad (32)$$

then

$$H = \begin{pmatrix} \mathbb{E}[h^2] & 0 \\ 0 & 1 \end{pmatrix}, \quad (33)$$

and the weight and bias directions are locally decoupled in this model.

For a vector hidden representation $h \in \mathbb{R}^m$, the same structure becomes

$$H = \begin{pmatrix} \mathbb{E}[hh^T] & \mathbb{E}[h] \\ \mathbb{E}[h]^T & 1 \end{pmatrix}. \quad (34)$$

Centering makes the weight–bias cross block vanish. It does *not* automatically decorrelate the hidden features from each other; the matrix $\mathbb{E}[hh^T]$ can still have substantial off-diagonal terms. Thus centering is useful but not equivalent to full whitening.

6.1 A numerical example

Let the uncentered hidden values be

$$h \in \{0.5, 1.5\}$$

with equal probability. Then

$$\mathbb{E}[h] = 1, \quad \mathbb{E}[h^2] = 1.25,$$

so

$$H_{\text{uncentered}} = \begin{pmatrix} 1.25 & 1 \\ 1 & 1 \end{pmatrix}.$$

Now center the same information:

$$\tilde{h} = h - 1 \in \{-0.5, +0.5\}.$$

Then

$$\mathbb{E}[\tilde{h}] = 0, \quad \mathbb{E}[\tilde{h}^2] = 0.25,$$

so

$$H_{\text{centered}} = \begin{pmatrix} 0.25 & 0 \\ 0 & 1 \end{pmatrix}.$$

The centered parameterization has removed the cross-term completely. In this example the approximate spectral condition number falls from about 18.2 to 4, illustrating why the corresponding quadratic loss can be easier for first-order gradient descent.

Connection between the two explanations. The Stanford zig-zag picture emphasizes per-example gradient signs. The Hessian argument emphasizes parameter coupling and conditioning over a data distribution. They are not identical arguments, but both arise because a nonzero activation mean makes downstream weight coordinates interact with the baseline.

7 Why random initialization strengthens the case for centering

Assume a conventional random initialization in which weights are drawn from a distribution symmetric about zero and initial biases are near zero. Early in training, preactivations are therefore often approximately symmetric around zero.

For the centered activation

$$g(z) = 2\sigma(z) - 1, \tag{35}$$

we have the odd-symmetry property

$$g(-z) = -g(z). \tag{36}$$

If the distribution of z is symmetric, this implies

$$\mathbb{E}[g(z)] = 0. \tag{37}$$

For the shifted activation

$$f(z) = g(z) + 1,$$

we instead get

$$\mathbb{E}[f(z)] = 1. \quad (38)$$

Thus, under the same initialization, the centered version naturally sends an approximately zero-mean signal to the next layer.

The difference can also be seen directly in the next preactivation. Let

$$h = \mathbf{1} + \tilde{h}, \quad \mathbb{E}[\tilde{h}] \approx 0.$$

Then

$$z_{\text{next}} = Wh + b = W\tilde{h} + W\mathbf{1} + b. \quad (39)$$

The term $W\mathbf{1}$ acts like an additional random offset. Across random initializations its expectation may be zero, but in any one initialized network it is generally nonzero. Centering removes this particular offset term.

This observation aligns with the historical analysis of Glorot and Bengio, who found that the nonzero mean of logistic-sigmoid activations is problematic in deep networks under random initialization and can contribute to saturation of upper layers [3].

8 Why this is a preference, not a universal law

Zero-centered hidden activations are often a better default in the narrow comparison considered here, but several qualifications matter.

8.1 Saturation can dominate centering

Both logistic sigmoid and tanh saturate. Their derivatives approach zero in the tails. A centered saturating activation may still train worse than a non-centered activation that avoids saturation. ReLU is the classic counterexample: it is not zero-centered, yet its positive branch has derivative one and historically proved much easier to optimize in many deep architectures.

8.2 Range does not guarantee empirical centering

A function with range $(-1, 1)$ is not automatically zero-mean in actual use. If most preactivations are positive, then its empirical activation mean may also be strongly positive.

8.3 Normalization can change the comparison

If a network explicitly normalizes internal representations, then the natural centering advantage of a $(-1, 1)$ activation may be reduced or partly replaced by the normalization mechanism. BatchNorm and LayerNorm explicitly subtract a mean; RMSNorm, notably, does not. These methods are summarized in section B.

8.4 Output layers have different semantics

For binary classification, a logistic sigmoid is useful precisely because its output lies in $(0, 1)$ and can be interpreted as a Bernoulli probability. The zero-centering argument is primarily about *hidden representations*, not a rule that all network outputs should be centered.

9 Practical rule of thumb

For two otherwise equivalent monotone S-shaped hidden activations that differ mainly by a constant vertical shift, under ordinary random initialization, with downstream biases and without normalization, the centered version is generally the preferable default.

A useful summary is:

$$\begin{array}{c} \text{(same function class)} \wedge \text{(same derivative shape)} \\ \wedge \text{(following-layer bias)} \\ \Downarrow \\ \text{centering changes optimization geometry, not expressivity} \end{array} \tag{40}$$

The most important benefits are:

- per-example gradients are not forced into only same-sign weight directions;
- the bias can represent the baseline while weights represent deviations around it;
- the weight–bias cross-coupling term disappears in the simple Hessian model when $\mathbb{E}[h] = 0$;
- symmetric random initialization naturally produces approximately zero-mean activations for an odd centered nonlinearity.

10 Recommended educational sources

The following sources complement one another rather than replacing one another.

1. **Stanford CS231n, Lecture 7: Training Neural Networks** [1]. Slides 29–33 give the clearest short visual explanation of the same-sign gradient restriction and the resulting zig-zag path, followed immediately by the observation that $\tanh$ is zero-centered.
2. **LeCun, Bottou, Orr, and Müller, “Efficient BackProp”** [2]. This is the strongest classical educational source for the broader optimization argument: centering, scaling, curvature, and convergence of backpropagation.
3. **Glorot and Bengio, 2010** [3]. This paper directly studies random initialization in deep feed-forward networks and explicitly identifies the nonzero mean of logistic-sigmoid activations as problematic.
4. **Goodfellow, Bengio, and Courville, *Deep Learning*** [4]. Chapters on numerical optimization and training deep models provide the broader language of ill-conditioning, curvature, initialization, and gradient propagation.

A Subdivision-based and spline-based saturating activation functions

The preceding discussion concerns centering, but it also motivates a broader question: must a sigmoid-like activation be an exponential function such as logistic sigmoid or tanh? No. Many monotone saturating curves can serve as nonlinear activations, although their optimization properties can differ substantially.

A.1 From corner cutting to smooth transitions

Chaikin’s corner-cutting algorithm repeatedly replaces a control polygon by a refined polygon obtained by cutting corners. The classical Chaikin scheme converges to a quadratic B-spline curve [6, 7]. This makes the following construction conceptually natural: begin with a step-like or clipped control shape, apply a subdivision rule, and obtain a smooth or piecewise-smooth saturating transition.

One should distinguish this intuition from a specific neural-network result. The 2024 work of López-Ureña does not merely say “apply Chaikin to a step.” It develops activation functions from the limit functions of convergent subdivision schemes and requires two structural properties: *refinability* and the ability to *sum the identity* [5]. These properties permit function-preserving widening and deepening of a neural network.

A.2 A quadratic spline activation

A representative centered spline activation presented by López-Ureña is

$$\sigma_{B^2}(t) = \begin{cases} -\frac{1}{2}, & t \leq -1, \\ t \left(1 - \frac{|t|}{2}\right), & -1 \leq t \leq 1, \\ \frac{1}{2}, & t \geq 1. \end{cases} \quad (41)$$

It is monotone and saturating, but unlike logistic sigmoid or tanh it reaches exact saturation at finite $|t|$.

Its derivative is especially simple:

$$\sigma'_{B^2}(t) = \begin{cases} 0, & |t| \geq 1, \\ 1 - |t|, & |t| < 1. \end{cases} \quad (42)$$

Thus the derivative is triangular. This is computationally simple, but exact saturation means that a neuron outside the transition interval has an exactly zero local derivative rather than merely a very small one.

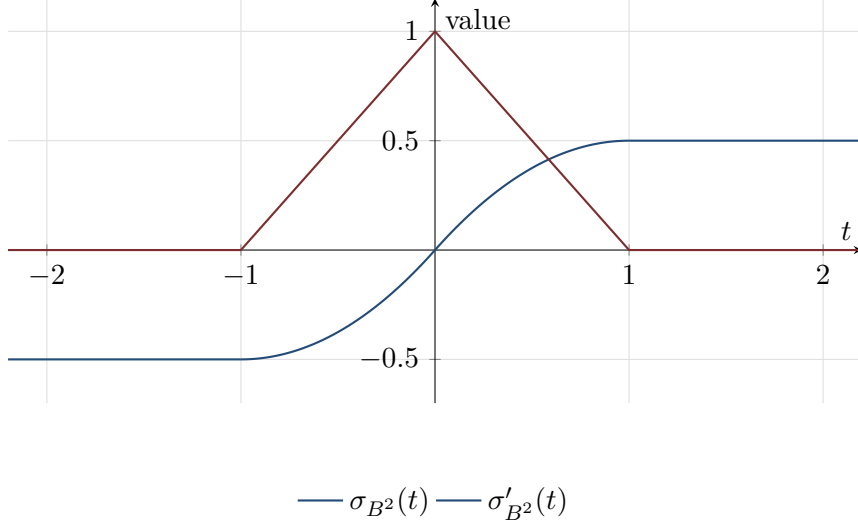


Figure 3: A subdivision-derived quadratic spline activation and its triangular derivative.

A.3 Why “without altering outcomes” matters

The same paper studies activations satisfying a refinement identity of the form

$$\sigma(t) = \sum_{\ell=0}^{A-1} a_{\ell} \sigma(2t + \tau - \ell), \quad (43)$$

which allows one neuron to be replaced by several scaled and shifted copies while preserving the function computed by the network. A second identity lets collections of activations sum to the identity map over an interval, allowing insertion of a new nonlinear layer without initially changing the network output [5].

The purpose is therefore not simply to invent another S-shaped curve. The subdivision structure provides algebraic identities that can preserve a trained function while the architecture is widened or deepened. After the transformation, training can continue in a larger parameter space.

B Normalization layers and their relation to centering

Modern deep networks frequently control internal signal statistics explicitly. This reduces the extent to which an activation function must provide centering by itself.

B.1 Batch Normalization

Batch Normalization (BatchNorm) normalizes a feature using statistics computed across a minibatch [8]:

$$\hat{z} = \frac{z - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}}, \quad y = \gamma \hat{z} + \beta. \quad (44)$$

It explicitly recenters and rescales the signal, then restores learnable scale and offset through γ and β . Its drawbacks include dependence on batch statistics, sensitivity to small batches, and a distinction between training-time and inference-time behavior.

B.2 Layer Normalization

Layer Normalization (LayerNorm) computes mean and variance across features of a single example rather than across a minibatch [9]:

$$\hat{z}_j = \frac{z_j - \mu_{\text{layer}}}{\sqrt{\sigma_{\text{layer}}^2 + \epsilon}}, \quad y_j = \gamma_j \hat{z}_j + \beta_j. \quad (45)$$

It therefore does not depend on which other examples happen to be in the minibatch and performs the same basic normalization at training and inference time.

B.3 RMSNorm: an important qualification to the centering story

RMSNorm deliberately removes the mean-subtraction step and normalizes primarily by root-mean-square magnitude [10]:

$$\text{RMS}(z) = \sqrt{\frac{1}{m} \sum_{j=1}^m z_j^2 + \epsilon}, \quad y_j = \gamma_j \frac{z_j}{\text{RMS}(z)}. \quad (46)$$

The success of RMSNorm is an important caution against overgeneralizing the slogan “zero mean is always necessary.” Re-centering is often useful, but modern optimization also depends critically on controlling scale, residual pathways, initialization, and the architecture as a whole.

B.4 What normalization changes in the $(0, 2)$ vs. $(-1, 1)$ comparison

If a normalization layer explicitly subtracts the mean after a hidden transformation, then much of the natural centering advantage of a $(-1, 1)$ activation can disappear. In a plain multilayer perceptron without normalization, activation centering directly changes the signal received by the next layer. In a normalized architecture, that signal may be recentered or rescaled before the next transformation.

This is why the classical tanh-versus-sigmoid centering discussion is most transparent in unnormalized feed-forward networks. Modern architectures still care deeply about conditioning and stable signal propagation, but they may achieve those goals using normalization, residual connections, careful initialization, or combinations of these techniques rather than relying on the activation function alone.

References

- [1] F.-F. Li, Y. Li, and R. Gao, *CS231n: Deep Learning for Computer Vision, Lecture 7: Training Neural Networks*, Stanford University, April 25, 2023. https://cs231n.stanford.edu/slides/2023/lecture_7.pdf.

- [2] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, “Efficient BackProp,” in *Neural Networks: Tricks of the Trade*, Lecture Notes in Computer Science, vol. 1524, Springer, 1998. <https://leon.bottou.org/papers/lecun-98x>.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 9:249–256, 2010. <https://proceedings.mlr.press/v9/glorot10a.html>.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. <https://www.deeplearningbook.org/>.
- [5] S. López-Ureña, “Activation functions enabling the addition of neurons and layers without altering outcomes,” arXiv:2410.12625, first posted 2024; revised version 2025. <https://arxiv.org/abs/2410.12625>.
- [6] G. M. Chaikin, “An algorithm for high speed curve generation,” *Computer Graphics and Image Processing*, vol. 3, pp. 346–349, 1974.
- [7] Caltech Multi-Res Modeling Group, “Chaikin’s Algorithm: Drawing Smooth Curves through Corner Cutting.” <https://multires.caltech.edu/teaching/demos/java/chaikin.htm>.
- [8] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” arXiv:1502.03167, 2015. <https://arxiv.org/abs/1502.03167>.
- [9] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” arXiv:1607.06450, 2016. <https://arxiv.org/abs/1607.06450>.
- [10] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” arXiv:1910.07467, 2019. <https://arxiv.org/abs/1910.07467>.